MODULE 04 · DOMAIN 1 - DATA PREPARATION FOR ML AND AI

Choosing a transformation engine

Glue, EMR, Processing jobs, DataBrew, Data Wrangler, Lambda. Which, and why.

MODULE 04 of 43 DOMAIN 1 - Data Prep WEIGHT 28%

admin@sthithapragna.com

Almost every engine **can** do the job. The question is always which one **should**.

What 'transformation' covers in Domain 1

01
Clean
Outliers, missing values, duplicates, bad types.

›

02
Reshape
Join, aggregate, pivot, flatten nested records.

›

03
Engineer
Encode, scale, bin, split, derive new features.

›

04
Land
Write in the right format, partitioned, ready to train.

ON THE EXAM

Module 05 covers the techniques in the third box. This module is only about which engine runs them.

Six engines, and the one-line version of each

ENGINE	THE ONE-LINE VERSION
AWS Glue	Serverless Spark. The default for ETL at scale
Amazon EMR	Managed Spark, Hive, Presto clusters. Control and scale
SageMaker Processing	Run your own container as a job, inside the ML workflow
AWS Glue DataBrew	Visual, no-code recipes. For analysts, not engineers
Data Wrangler in Canvas	Visual ML data prep, exports to a pipeline
AWS Lambda	Small, fast, per-record or per-file transforms

ON THE EXAM

Athena and Redshift also transform data with SQL. They are covered where they belong, in the query modules.

AWS Glue, and why it is usually the answer

- **Serverless Spark.** No cluster to size, start, patch or shut down.
- Billed by **DPU-hour** while the job runs. Nothing runs, nothing is billed.
- Integrated with the **Glue Data Catalog**, so schemas are discovered and shared.
- **Job bookmarks** track what has already been processed, so reruns are incremental.
- **Glue 6.0** became generally available in August 2026.

> ON THE EXAM
>
> 'LEAST operational overhead' plus 'transform data at scale' is the Glue question, and it is very common.

Job types, and what each is for

JOB TYPE	USE IT FOR
Spark	Distributed ETL over large datasets. The main one
Spark Streaming	Continuous transformation of a Kinesis or Kafka stream
Python shell	Small, single-node Python. No Spark overhead at all
Ray	Closed to new customers since 30 April 2026

ON THE EXAM

Python shell jobs matter: a small job does not need a Spark cluster, and paying for one is a wrong answer.

DPUs, workers and the cost levers

- A **DPU** is Glue's capacity unit. You choose worker type and number of workers.
- **G.1X, G.2X, G.4X, G.8X** scale memory and CPU per worker for heavier jobs.
- **Auto scaling** adds and removes workers during the job instead of fixing the count.
- **Flex execution** runs on spare capacity at lower cost, for jobs with no deadline.
- Over-provisioning DPUs is the most common Glue cost mistake. Idle executors still bill.

> ON THE EXAM
>
> Flex is the Spot equivalent for Glue: cheaper, and the job may start later. Deadlines rule it out.

The Glue Data Catalog and crawlers

- A central **metadata store**: databases, tables, schemas, partitions. Not the data itself.
- **Crawlers** scan S3, infer schema and partitions, and register tables automatically.
- One catalog serves **Athena, EMR, Redshift Spectrum and Glue**. Define once, query anywhere.
- Crawlers cost money per run. Scheduling one hourly over static data is waste.
- You can register schemas manually or from a job, so a crawler is not compulsory.

> ON THE EXAM
>
> The catalog is why Glue is the centre of gravity for data on AWS, even when another engine does the work.

Triggers, workflows and bookmarks

- **Triggers** start jobs on a schedule, on an event, or when another job succeeds.
- **Workflows** chain crawlers and jobs into one visual graph with shared state.
- **Job bookmarks** record what was processed, so a rerun handles only new data.
- Bookmarks are per job. Rename or recreate the job and the bookmark starts empty.
- For anything crossing service boundaries, Step Functions is the better orchestrator.

> ON THE EXAM
>
> Glue workflows are fine inside Glue. The moment a pipeline spans Glue, SageMaker and Lambda, use Step Functions.

Incremental processing and idempotency

- **Full reprocess** is simple and correct, and gets expensive as history grows.
- **Incremental** processes only new partitions or bookmarked files. Cheap, and fiddlier.
- Write to a **partition path** and overwrite that partition, so a rerun replaces rather than duplicates.
- Appending on rerun is how duplicate rows enter a training set unnoticed.
- Late-arriving data is the reason to reprocess recent partitions on a schedule.

> ON THE EXAM
>
> A job that produces a different result when run twice will eventually be run twice. Design for it.

Three ways to author, and one to remember

- **Glue Studio.** Visual drag-and-drop that generates the underlying Spark code.
- **Scripts.** PySpark or Scala written directly, for anything non-trivial.
- **Interactive sessions.** Notebook-style development against real Glue infrastructure.
- **DynamicFrames** are Glue's own abstraction. They tolerate schema drift better than DataFrames.
- You can convert between DynamicFrame and Spark DataFrame freely inside a job.

> ON THE EXAM
>
> DynamicFrame is the Glue-specific concept most likely to appear. It handles inconsistent schemas without failing the job.

Amazon EMR, and what it buys you

- Managed clusters running **Spark, Hive, Presto, Trino, HBase, Flink** and more.
- **You choose the version.** Specific Spark releases, specific libraries, custom bootstrap.
- **Spot Instances** on task nodes can cut cost dramatically for long-running work.
- Best economics at **sustained heavy scale**, where serverless per-second billing adds up.
- You now own cluster sizing, tuning, patching and the operational burden that comes with it.

> ON THE EXAM
>
> EMR is chosen for control or for scale economics. It is almost never chosen for convenience.

Node types, fleets and managed scaling

- **Primary** node coordinates. **Core** nodes hold HDFS data. **Task** nodes compute only.
- Task nodes carry no data, so they are the safe place for **Spot Instances**.
- Spot on core nodes risks losing HDFS blocks. This is a classic wrong answer.
- **Instance fleets** mix types and pricing with automatic fallback when Spot is unavailable.
- **Managed scaling** resizes the cluster automatically against the workload.

> ON THE EXAM
>
> Spot on task nodes, On-Demand on core. That split is examinable and it appears almost verbatim.

EMR on EC2, on EKS, and Serverless

- **EMR on EC2.** The classic cluster. Maximum control, maximum operational work.
- **EMR on EKS.** Runs Spark on an existing Kubernetes cluster, sharing capacity.
- **EMR Serverless.** No cluster at all. Submit a job, it provisions and releases.
- EMR Serverless narrows the gap with Glue while keeping EMR's framework versions.
- Instance fleets mix On-Demand and Spot with fallbacks, for resilient cheap clusters.

> ON THE EXAM
>
> EMR Serverless changes an old exam answer. 'EMR means managing a cluster' is no longer automatically true.

AWS Glue or Amazon EMR?

GLUE, WHEN

You want it to just run

- Least operational overhead is stated.
- Standard Spark ETL, no exotic libraries.
- Intermittent or scheduled jobs.
- The Data Catalog matters to you.

EMR, WHEN

You need control or scale

- A specific framework or version is required.
- Hive, Presto, HBase or custom bootstrap.
- Sustained heavy load, Spot economics matter.
- A team already skilled in cluster tuning.

THE TIEBREAKER

Least operational overhead points to Glue. A named framework version, or heavy sustained scale, points to EMR.

SageMaker Processing jobs

- Runs **your container** on managed infrastructure: read from S3, process, write to S3.
- Built-in containers for **scikit-learn, PySpark, PyTorch** and more.
- **The same code and libraries as training.** That is the point, and it prevents skew.
- A native **step in SageMaker Pipelines**, so it fits the ML workflow without glue code.
- Also used for evaluation and batch scoring, not only preprocessing.

> ON THE EXAM
>
> Same libraries at train time and transform time is the argument. Reimplementing a transform in Spark is how skew starts.

What you actually specify

SETTING	WHAT IT CONTROLS
`Container image`	Built-in scikit-learn, PySpark, or your own from ECR
`Instance type and count`	More than one instance means distributed processing
`ProcessingInput`	S3 location mounted into the container at a local path
`ProcessingOutput`	Local path copied back to S3 when the job finishes
`S3DataDistributionType`	FullyReplicated to every instance, or ShardedByS3Key

ON THE EXAM

ShardedByS3Key is what makes a Processing job scale. FullyReplicated on a large dataset just copies it everywhere.

AWS Glue or SageMaker Processing?

GLUE, WHEN

It is a data engineering job

- Serving many consumers, not just one model.
- Joining and aggregating source systems.
- The output is a curated dataset or table.
- Data engineers own the pipeline.

PROCESSING, WHEN

It is a model's preprocessing

- The step belongs to one model's workflow.
- Same libraries as the training script.
- It runs inside a SageMaker Pipeline.
- ML engineers own the code.

THE TIEBREAKER

Ask who consumes the output. Many consumers means Glue. One model means Processing.

AWS Glue DataBrew

- Visual data preparation with **over 250 built-in transformations**. No code at all.
- Work is saved as a **recipe**, then applied to new data as a scheduled recipe job.
- **Profile jobs** produce summary statistics and a correlation matrix automatically.
- Aimed at **analysts and domain experts** who understand the data but do not write Spark.
- AWS publishes a migration path from DataBrew to Glue Studio. Note it, but DataBrew is current.

> ON THE EXAM
>
> 'The team has no engineers' or 'business analysts must maintain it' is the DataBrew signal.

Data Wrangler, now inside SageMaker Canvas

- Visual data preparation aimed at **ML specifically**: encoding, balancing, feature creation.
- **It lives in SageMaker Canvas now**, not standalone Studio Classic. Flows import across.
- 300+ transforms and 50+ data sources, including S3, Redshift and Snowflake.
- **Exports to a Processing job, a Pipeline or Python code.** That is what separates it from DataBrew.
- Older material says 'Data Wrangler in Studio'. The location changed; the capability did not.

> ON THE EXAM
>
> Data Wrangler produces something a pipeline can run. Prototype visually, then export and automate.

DataBrew or Data Wrangler?

DATABREW, WHEN

General data quality

- Cleaning for analytics as well as ML.
- Analysts own it, and always will.
- Profiling and data quality is the goal.
- Output feeds many consumers.

DATA WRANGLER, WHEN

Preparing features for a model

- ML-specific transforms: encoding, balancing.
- You will export it into a pipeline.
- It feeds training, and only training.
- ML engineers will take it over.

THE TIEBREAKER

DataBrew ends in a cleaned dataset. Data Wrangler ends in a pipeline step. That difference decides most questions.

AWS Lambda, and where it stops

- Ideal for **per-file or per-record** work triggered by an S3 event or a stream.
- Starts in milliseconds. No cluster, no job submission, no warm-up.
- **15-minute maximum** execution. A long transformation simply cannot finish.
- Memory-bounded, and it does not distribute. One invocation, one worker.
- Firehose can invoke Lambda inline to transform records as they pass through.

> ON THE EXAM
>
> Small and event-driven is Lambda. Large or distributed is not, no matter how neat the trigger would be.

Flattening nested records

- JSON from APIs and streams arrives nested. Most ML algorithms want flat columns.
- **Glue relationalize** flattens a nested DynamicFrame into related flat frames.
- In Spark, **explode** turns an array into one row per element.
- Athena can query nested data directly with struct and array syntax, no flattening needed.
- Exploding a large array multiplies row count fast. Watch the output size.

> ON THE EXAM
>
> Nested data is not a problem to be fixed reflexively. Flatten it when the model needs columns, not before.

Four Spark concepts that appear

- **Partitions** are the unit of parallelism. Too few and cores idle; too many and overhead dominates.
- **Shuffle** moves data between executors on joins and groupBy. It is the expensive operation.
- **Data skew:** one key holds most rows, so one task runs long after the rest finish.
- **Broadcast join** sends a small table to every executor, avoiding the shuffle entirely.
- Caching a DataFrame reused several times avoids recomputing the whole lineage.

> ON THE EXAM
>
> A Spark job where 199 tasks finish quickly and one runs for an hour is skew, not an undersized cluster.

Transformation jobs, and how they fail

SYMPTOM	USUAL CAUSE AND FIX
Executor out of memory	A skewed key, or collect() pulling data to the driver. Repartition
One task runs far longer than the rest	Data skew. Salt the key, or broadcast the small side
Job succeeds but output is thousands of tiny files	Too many output partitions. Coalesce before writing
Rerun produces duplicate rows	Appending instead of overwriting a partition
Glue job reprocesses everything each night	Job bookmarks not enabled, or the job was recreated
Costs rise while data volume is flat	Over-provisioned DPUs, or a crawler running too often

ON THE EXAM

Most of these are visible in metrics before anyone complains. Module 31 covers where to look.

How you write is part of the transformation

- Write **Parquet, partitioned** on the column downstream queries filter by. Module 03's lesson.
- Aim for output files of **tens to hundreds of MB**, not kilobytes and not gigabytes.
- **Coalesce or repartition** before writing to control how many files you produce.
- A transform that fixes the data and leaves 200,000 tiny files has moved the problem downstream.
- Register the output in the Data Catalog so the next team can find it.

> ON THE EXAM
>
> The job is not finished when the data is correct. It is finished when the next consumer can read it efficiently.

How each engine bills you

ENGINE	COST SHAPE
AWS Glue	Per DPU-hour while running, 1-minute minimum. Idle costs nothing
Glue Flex	Same, discounted, on spare capacity. Start time is not guaranteed
EMR on EC2	Per instance-hour for the whole cluster, including idle time
EMR Serverless	Per vCPU and GB-hour consumed by the job only
SageMaker Processing	Per instance-hour for the job duration only
DataBrew	Per session for interactive work, per node-hour for recipe jobs
Lambda	Per request and per GB-second. Nothing between invocations

ON THE EXAM

A cluster you forgot to terminate is the single most common surprise on an ML data bill.

admin@sthithapragna.com

SQL engines doing transformation work

APPROACH	WHEN IT IS THE RIGHT ANSWER
Athena CTAS	Create a new table from a query. Reformat and repartition in one statement
Athena UNLOAD	Write query results to S3 in Parquet or ORC directly
Redshift SQL	The data is already in Redshift and the transform is relational
Redshift Spectrum	Join warehouse tables to S3 data without loading it first

ON THE EXAM

If the transformation is expressible as SQL and the data is already queryable, a Spark job may be the heavy answer.

Signal to engine

WHAT THE SCENARIO SAYS	WHAT TO REACH FOR
Large-scale ETL, least operational overhead	AWS Glue
Specific Spark or Hive version, custom libraries	Amazon EMR
Sustained heavy load, Spot economics matter	Amazon EMR
Same libraries as training, inside a pipeline	SageMaker Processing
Analysts with no coding skills	AWS Glue DataBrew
Visual ML feature prep, then automate it	Data Wrangler in Canvas
Per-file transform on arrival, seconds not minutes	AWS Lambda
It is really just SQL over queryable data	Athena CTAS

ON THE EXAM

Read for three things: who does the work, how big the data is, and whether it lives in an ML pipeline.

Training and serving skew

- Training features built in Spark, inference features built in application code.
- The two implementations drift apart. Accuracy in production falls below test accuracy.
- Nothing errors. There is no alarm for 'these two functions disagree'.
- **Fixes:** one implementation shared by both paths, or the Feature Store from module 06.
- This is the strongest argument for SageMaker Processing over a separate ETL engine.

> ON THE EXAM
>
> A model that scored well in testing and worse in production, with no code change, is often skew rather than drift.

Testing a transformation

- **Row counts in and out.** An unexpected drop usually means a join lost rows.
- **Null rates per column** before and after. A new null column is a silent failure.
- **Run it twice on the same input** and compare. Different output means it is not idempotent.
- **Glue Data Quality** expresses rules as code and fails the job when they break.
- DataBrew profile jobs give you the same statistics visually, without writing rules.

> ON THE EXAM
>
> Transformation bugs do not raise errors. They produce plausible data that trains a slightly worse model.

Match the engine to the **team and the workflow**, not to the size of the data alone.

> A marketing team receives a 40 GB CSV extract each week. Before it can be used, columns must be renamed, currency strings converted to numbers, duplicates removed and outliers flagged.
>
> The team consists of business analysts. Nobody writes Python or Spark, and the company does not want to allocate an engineer to maintain this permanently.

Which approach fits best?

A. An AWS Glue Spark job authored in Glue Studio and scheduled weekly

B. An AWS Glue DataBrew recipe, applied by a scheduled recipe job

C. An Amazon EMR cluster running a weekly Spark job

D. A SageMaker Processing job running a scikit-learn script

◆ PAUSE AND DECIDE

Which approach fits best?

A. An AWS Glue Spark job authored in Glue Studio and scheduled weekly

The closest wrong answer. Studio is visual, but it generates Spark code that someone must own when a transform needs changing.

B. An AWS Glue DataBrew recipe, applied by a scheduled recipe job

Built for exactly this: analysts, no code, over 250 transforms, saved as a reusable recipe and scheduled against new data.

C. An Amazon EMR cluster running a weekly Spark job

Most operational overhead of the four, for 40 GB once a week. It needs the engineer the company explicitly will not provide.

D. A SageMaker Processing job running a scikit-learn script

Requires Python, and this is general data cleaning rather than one model's preprocessing. Wrong tool and wrong owner.

THE SIGNAL

Two constraints decide it: no coding skills, and no engineer to maintain it. Both are about people, not about data.

A genomics company runs a nightly pipeline over roughly 30 TB. The processing depends on a specific Apache Spark release and three custom native libraries installed through bootstrap actions.

The cluster runs for six to eight hours every night. Cost is a stated priority, and the team already includes engineers experienced in Spark tuning.

Which engine should they use?

A AWS Glue Spark jobs with auto scaling enabled

B AWS Glue Spark jobs using Flex execution to reduce cost

C SageMaker Processing jobs using the PySpark container

D Amazon EMR on EC2 with Spot Instances on task nodes

◆ PAUSE AND DECIDE

Which engine should they use?

A AWS Glue Spark jobs with auto scaling enabled

Glue does not let you pin an arbitrary Spark release or run bootstrap actions for native libraries. The requirement is not satisfiable.

B AWS Glue Spark jobs using Flex execution to reduce cost

Same blocker as A, plus Flex delays start until spare capacity appears, which suits a nightly deadline badly.

C SageMaker Processing jobs using the PySpark container

This is data engineering serving a pipeline, not one model's preprocessing. It also inherits the same version constraint problem.

D Amazon EMR on EC2 with Spot Instances on task nodes

Version pinning and bootstrap actions are EMR's defining capability. Six to eight hours nightly at 30 TB is where Spot economics pay.

THE SIGNAL

A named framework version plus custom native libraries is the EMR signal. It is one of the few genuinely hard constraints in Domain 1.

A team trains a churn model with a scikit-learn script. Feature scaling and categorical encoding currently run in a separate AWS Glue job, reimplemented in PySpark by the data engineering team.

Production accuracy is consistently below validation accuracy. Investigation shows the Spark encoding and the scikit-learn encoding handle unseen categories differently. The team wants preprocessing inside their SageMaker Pipeline.

What should they move to?

A Rewrite the Glue job to match the scikit-learn behaviour exactly, and add tests

B Move preprocessing into AWS Glue DataBrew so both teams can see the transforms

C Move preprocessing into a SageMaker Processing job using the scikit-learn container

D Move preprocessing into the training script itself, so it runs on every training job

◆ PAUSE AND DECIDE

What should they move to?

A. Rewrite the Glue job to match the scikit-learn behaviour exactly, and add tests

Keeps two implementations of one transform and promises to hold them in sync forever. Tests reduce the odds; they do not remove the cause.

B. Move preprocessing into AWS Glue DataBrew so both teams can see the transforms

Improves visibility and changes nothing about the mismatch. It is a third implementation, and it cannot use scikit-learn's encoder.

C. Move preprocessing into a SageMaker Processing job using the scikit-learn container

One implementation, the same library as training, and a native Pipeline step. It removes the cause rather than managing it.

D. Move preprocessing into the training script itself, so it runs on every training job

Fixes training but not inference, and reprocesses the full dataset on every run. Inference still needs the transform from somewhere.

THE SIGNAL

Two implementations of one transform is the disease. Any option that leaves two in place is treating a symptom.

> An IoT platform receives several thousand small JSON files into Amazon S3 each hour, each around 200 KB. Each file needs its timestamps normalised and three fields redacted before it lands in the curated zone.
>
> The transform is simple and independent per file. The team wants records available within seconds of arrival, with the LEAST operational overhead.

Which approach fits?

A. An AWS Lambda function triggered by S3 event notifications

B. An AWS Glue Spark job scheduled every five minutes

C. An EMR Serverless application submitting a job per batch of files

D. A SageMaker Processing job triggered by an EventBridge rule

◆ PAUSE AND DECIDE

Which approach fits?

A An AWS Lambda function triggered by S3 event notifications

Small, independent, per-file work with a seconds-level requirement. Lambda starts in milliseconds and scales out per file automatically.

B An AWS Glue Spark job scheduled every five minutes

Job start-up alone breaks the seconds requirement, and a distributed engine is being paid for to edit 200 KB files independently.

C An EMR Serverless application submitting a job per batch of files

Removes the cluster but keeps the submission overhead and the batching. More machinery than a field redaction needs.

D A SageMaker Processing job triggered by an EventBridge rule

Processing jobs are for ML preprocessing, and each launches infrastructure. Thousands per hour is the wrong shape entirely.

THE SIGNAL

Per-file, independent, and seconds. Three signals, all pointing away from any engine that has to start up first.

What to take from this module

- **Glue is the default.** Serverless Spark, Data Catalog, bookmarks, least overhead.
- **EMR for control or scale.** A named version or custom libraries is a hard constraint.
- **Processing for ML preprocessing.** Same libraries as training, native pipeline step.
- **DataBrew for analysts. Data Wrangler for ML features**, and it lives in Canvas now.
- **Ask who consumes the output** and who maintains the code. Data size rarely decides it.

> ON THE EXAM
>
> Module 05 moves from which engine to which transformation: encoding, scaling, binning and the rest.